Lesson plan · for the teacher

The school bus route

Optimization · Level 3 · Lesson 3

Time	Materials
40 min	The playground or a large room; Chalk to draw the paths; A card with each village's name

Learning goals

- See why trying every bus route is impossible.
- Build a route with nearest neighbour and improve it with 2-opt.
- Judge a route by its length and by the longest ride.

Activity: The human bus route

1. Warm-up (5 min): Place six pupils as villages with name cards and mark the school door. Each pupil guesses the shortest round trip by looking and writes it down.
2. Nearest neighbour (10 min): One pupil is the bus. From the school door, it always walks to the nearest village not yet visited, then back. The class counts the steps of each leg; write them on the board and add them up.
3. Untangle (10 min): Draw the route in chalk. Find two paths that cross, reverse the stretch between them and walk it again. Is it shorter? Keep going until no swap helps.
4. Longest ride (10 min): For each village, add the steps from its pick-up to the school. Which village rides longest on each route?
5. Wrap-up (5 min): Compare the routes by total steps and by the longest ride. Ask which the class would choose, and what rule would make it fair.

Spread the villages unevenly, with one far corner, so nearest neighbour leaves a long walk home. No playground? Use desks and a floor plan drawn on the board.

Discussion

- Why was the walk home so long with nearest neighbour?
- Did swapping two crossing paths always help?
- Which village rode longest, and is that fair?

Exit ticket

- Draw a route with a crossing, and the shorter route after one 2-opt swap.

Try it at school tomorrow

On the playground, pupils stand as villages and one pupil is the bus, starting at the school door. Drive nearest neighbour, counting steps. Then find two paths that cross, swap them, and count again. Finish by asking who rode the longest, and how you would make that fairer.

Practice questions · for you

The school bus route

Optimization · Level 3 · Lesson 3

1 **Route A is 2 km shorter, but one village rides 50 minutes; on B nobody rides over 35. How should we choose?**

- ◯ Always the shortest: fuel comes first
- ◯ Agree a longest-ride limit with families, then the shortest route that keeps it
- ◯ Let the driver decide each day

2 **Put the nearest-neighbour rule in order.**

- ☐ Find the closest village not yet visited
- ☐ Drive there and add it to the route
- ☐ Drive back to the school
- ☐ Start at the school
- ☐ Repeat until every village is in the route

3 **One more village roughly doubles the number of possible bus routes.**

◯ True ◯ False

4 **d[a][b] is the distance in km between places a and b; place 0 is the school. Write nearest neighbour: starting at the school, keep adding the closest place not yet in route until all 5 places are in. The program then prints the route and its length, including the drive back.**

```
d = [[0, 3, 3, 2, 6],
     [3, 0, 6, 4, 6],
     [3, 6, 0, 3, 7],
     [2, 4, 3, 0, 4],
     [6, 6, 7, 4, 0]]
route = [0]
# add the closest place not yet in route,
# until all 5 places are in

km = sum(d[a][b] for a, b in zip(route, route[1:] + [0]))
print(route, km)
```

Write your program:

5 **Put the lines in order: one pass of 2-opt that tries every pair of cut points and keeps a reversed stretch when it makes the route shorter.**

Number the lines in order (1, 2, 3…), and draw one arrow → in front of a line for each indent it needs. Cross out any line that doesn't belong.

- [ ] `if length(new) < length(route):`
- [ ] `if length(new) > length(route):`
- [ ] `route = new`
- [ ] `for i in range(1, len(route) - 1):`
- [ ] `new = route[:i] + mid + route[j + 1:]`
- [ ] `mid = list(reversed(route[i:j + 1]))`
- [ ] `for j in range(i + 1, len(route)):`

✎ **Think of a route at your school: the bus, a delivery, a walk between buildings. What does it make short, and who pays for that?**

Check your answers

The school bus route

Optimization · Level 3 · Lesson 3

1 **Agree a longest-ride limit with families, then the shortest route that keeps it**

The total length and the longest ride are two goals.

2 **Top to bottom: 2, 3, 5, 1, 4**

Nearest neighbour only ever looks one step ahead: the closest unvisited village from where the bus is now.

3 **False**

It multiplies them by the new number of villages: 9 villages have 181,440 routes, 10 have 1,814,400, ten times as many.

4

```
d = [[0, 3, 3, 2, 6],
     [3, 0, 6, 4, 6],
     [3, 6, 0, 3, 7],
     [2, 4, 3, 0, 4],
     [6, 6, 7, 4, 0]]
route = [0]
while len(route) < 5:
    last = d[route[-1]]
    left = [v for v in range(5) if v not in route]
    route.append(min(left, key=lambda v: last[v]))
km = sum(d[a][b] for a, b in zip(route, route[1:] + [0]))
print(route, km)
```

Nearest neighbour gives 0 → 3 → 2 → 1 → 4 → 0, 23 km, and the first village (3) rides 21 of them.

5 **Top to bottom: 5, X, 6, 1, 4, 3, 2**

```
for i in range(1, len(route) - 1):
    for j in range(i + 1, len(route)):
        mid = list(reversed(route[i:j + 1]))
        new = route[:i] + mid + route[j + 1:]
        if length(new) < length(route):
            route = new
```

Two loops choose where to cut; the stretch between the cuts is reversed and glued back; the new route is kept only if it is shorter.